



MEMORIA TECNICA

Mantenimiento y Gestión del cambio en sistemas software



José Carrasco Haro
Jesús Bogarín Hernández
Jaime García Marín

Índice

1.	Introducción.....	2
1.1	Propósito del proyecto.....	2
1.2	Alcance y Funcionalidades principales	2
2.	Decisión de arquitectura y Diseño.	2
2.1	Arquitectura Hexagonal (puertos y adaptadores).....	2
2.2	Decisiones técnicas relevantes.....	3
2.2.1	Uso de DTOs para la comunicación.....	3
2.3	Desarrollo de Software.....	3
3.	Análisis de Deuda técnica y Comparativas de métricas.	3
3.1	Estado Inicial vs Estado actual.....	3
3.2	Identificación y Resolución de Deuda Técnica.....	4
3.3	Estrategia Testing y cobertura.....	4
3.3.1	Resultados de Cobertura.....	4
4.	Instrucciones de Instalación (Desde Release de GitHub)	5
4.1	Prerrequisitos.	5
4.2	Método 1: Ejecución mediante Docker	5
4.3	Método 2: Ejecución Local mediante .jar.....	5
5.	Demo Funcional.....	5
5.1	Descripción de la interfaz web.....	5
5.2	Flujo Principal de uso.....	6

1. Introducción

1.1 Propósito del proyecto

El sistema es una plataforma web diseñada para administrar y trazar el ciclo de vida de las incidencias técnicas. Su objetivo principal es organizar el trabajo mediante transacciones de estado (abierta, en_proceso, cerrada), además manteniendo un historial de cada petición.

El proyecto tiene como propósito implementar una arquitectura hexagonal con puertos y adaptadores. Este enfoque consigue aislar la lógica de dominio de los framework como springboot y la base de datos PostgreSQL o JPA, garantizando un sistema tolerante a cambios.

1.2 Alcance y Funcionalidades principales

El sistema se centra en tres entidades: Cliente, Técnico y Solicitudes. Sus funcionalidades se dividen en:

- **Administrador:**
 - Registra y da de alta a nuevos clientes y técnicos en el sistema.
 - Crea solicitudes asociada a un cliente.
 - Asigna incidencias a un solo técnico activo.
 - Reabre las solicitudes previamente cerradas.
- **Técnicos**
 - Visualiza las incidencias asignadas a cada técnico.
 - Capaz de cambiar el estado de una solicitud al iniciar los trabajos.
 - Cierre de solicitudes, apoyado con una regla de negocio que impide finalizar la incidencia sin haber pasado antes por el estado de proceso.
- **Características:**
 - **Auditoria:** Mapeo y registro cronológico de todos los cambios de estado que sufre una solicitud.
 - **API REST:** Endpoints expuestos mediante controladores, utilizando DTOs (records) para la transferencia de datos.
 - **Interfaz:** Frontend básico desarrollado en HTML, CSS y JS que interactúa asíncronamente con la API.

2. Decisión de arquitectura y Diseño.

2.1 Arquitectura Hexagonal (puertos y adaptadores)

El proyecto implementa una arquitectura hexagonal, cuyo principio fundamental es separar la lógica de negocio de cualquier infraestructura. Las dependencias fluyen de afuera hacia adentro.

- **Capa de dominio (com.mgcss.domain):** Es el núcleo del sistema. Contiene las entidades puras (solicitud, cliente, técnico) que encapsulan atributos y las reglas de negocio. Asimismo, define los puertos de salidas que son las interfaces (SolicitudRepository, TecnicoRepository, ClienteRepository) estos no revelan como se almacena la información.
- **Capa Servicios (com.mgcss.servide):** Actúa como los casos de uso de la aplicación. Los servicios (SolicitudService, ClienteService, TecnicoService) coordinan la lógica de las operaciones invocando a las entidades y comunicándose con los puertos de salida.
- **Capa de infraestructura (com.mgcss.infraestructura.persistente):** Funciona como el adaptador de salida, implementa los puertos del dominio a través de clases adaptadoras (SolicitudRepositoryAdapter, TecnicoRepositoryAdapter, ClienteRepositoryAdapter) Estas clases traducen los objetos de negocio en entidades físicas (SolicitudEntity, TecnicoEntity, ClienteEntity) y delegan la ejecución CRUD en la interfaces de JPA conectadas a PostgreSQL/H2.
- **Capa API (com.mgcss.api):** Funciona como el adaptador de entrada, contiene los controladores REST (SolicitudController, ClienteController, TecnicoController) encargados de exponer los endpoints públicos. Esta capa encapsula y valida la entrada de datos externos mediante (SolicitudRequestDTO) y transforman la salida del servicio mediante (SolicitudResponseDTO).

2.2 Decisiones técnicas relevantes

2.2.1 Uso de DTOs para la comunicación

Para el intercambio de datos entre la API y el cliente, se han implementado objetos de transferencia de datos utilizando java records.

- **Validación en la entrada:** Los DTOs como SolicitudoRequestDTO o ClienteRequestDTO encapsulan las reglas de validación. Esto permite interceptar datos corruptos o incompletos en la capa del controlador.
- **Desacoplamiento del dominio:** Evita exponer directamente las entidades de negocio al exterior, permitiendo modelar y limpiar la estructura enviada al frontend.

En lugar de utilizar excepciones se han utilizado excepciones de dominio personalizadas, aislando la lógica y mejorando la semántica del código.

- **Regla de negocio:** Se lanza cuando una operación viola una regla, como por ejemplo cuando se intenta asignar un técnico inactivo, esto está vinculado al código HTTP **400 bad request** mediante `@ResponseStatus`
- **EntidadNoEncontrada:** Se dispara desde los servicios de la aplicación cuando una búsqueda por identificador no tiene resultados en los repositorios, se asocia al código **404 not found**.

2.3 Desarrollo de Software

- **Java 17:** Es el lenguaje de programación principal del núcleo del sistema.
- **Spring Boot:** Framework central utilizado para dar soporte a la infraestructura de la aplicación.
- **PostgreSQL y H2 Database:** Durante el desarrollo local y la ejecución de pruebas automatizadas, se emplea la base de datos **H2**. En el entorno de producción, se utiliza PostgreSQL como el motor de base de datos relacional para asegurar la persistencia física y el aislamiento de los datos.
- **Docker:** El proyecto incluye un Dockerfile basado en una imagen permite levantar todo el entorno productivo (aplicación + base de datos PostgreSQL) con un único comando, controlando las variables de entorno.

3. Análisis de Deuda técnica y Comparativas de métricas.

3.1 Estado Inicial vs Estado actual.

Para evaluar y garantizar la calidad del código, el proyecto ha sido integrado en una plataforma de análisis continuo mediante SonarCloud y JaCoCo, A través de la sucesivas iteraciones y refactorizaciones se ha logrado transformar el estado del proyecto.

Métrica de Calidad (SonarCloud)	Estado Inicial	Estado Actual
Clasificación de Mantenibilidad	Grado B / C	A
Code Smells	> 15 smells	6
Deuda Técnica	2 horas	37 minutos
Índice de Endeudamiento	> 2.0%	0,1%
Insectos (Bugs / Fiabilidad)	Presencia de bugs menores	0 (Clasificación A)
Densidad de Duplicaciones	3.5%	0,0%
Cobertura General de Código	80%	86,3%
Pruebas Unitarias Ejecutadas	< 20 tests	63 tests (100% Éxito)

3.2 Identificación y Resolución de Deuda Técnica

A través del análisis proporcionado por SonarCloud, se identificaron varios problemas de diseño y malas prácticas en las fases iniciales del proyecto. Para solucionar y asegurar la mantenibilidad a largo plazo, se aplicaron técnicas dirigidas a dos bloques críticos:

A. Resolución de “Code Smells”

- **Problema:** El sistema utilizaba excepciones genéricas de java, en la lógica de dominio (solicitud) y en la capa de servicio (SolicitudServide). Esto provocaba inconsistencia en las aserciones de las pruebas unitarias.
- **Riesgo:** Dificultaba el manejo de errores en las capas superiores, los controladores REST no podían encontrar semánticamente el origen de la excepción.
- **Solución:** Se crearon las excepciones de dominio personalizadas (reglaDeNegocio y EntidadNoEcontrada) anotadas con sus correspondientes estados de respuesta HTTP, además se realizaron pruebas unitarias.

B. Eliminación de código muerto.

- **Problema:** En la estructura inicial del proyecto se detectó código redundante, específicamente en la clase cliente que se encontraba completamente vacía.
- **Riesgo:** Incrementaba innecesariamente la carga y afectaba de manera negativa las métricas reales de análisis de cobertura de código.
- **Solución:** Se implementó la clase cliente y se mapeó su relación con las solicitudes.

3.3 Estrategia Testing y cobertura.

Niveles y Tipos de Pruebas Implementados:

- **Pruebas Unitarias de Dominio:** Enfocadas en la lógica pura utilizando JUnit 5 sobre las entidades del núcleo (SolicitudTest, TecnicoTest, ClienteTest)
- **Pruebas de Aislamiento de Controladores (API):** Utilizan la anotación `@WebMvcTest` y la herramienta MockMvc para simular peticiones HTTP entrantes sobre la capa web.
- **Pruebas de Integración y Mapeo (Persistencia):** Validan el comportamiento de la capa de datos interactuando con una base de datos relacional H2 en memoria mediante `@DataJpaTest` para comprobar el almacenamiento real (JpaTecnicoRepositoryIntegrationTest).
- **Prueba de Carga de Contexto Global:** Un test de integración de extremo a extremo (MgcssTrackL2Grupo4ApplicationTests) configurado con `@SpringBootTest` que comprueba que el ecosistema completo de SpringBoot consiga levantar sin conflictos de dependencias ni fallos.

3.3.1 Resultados de Cobertura

- Se ejecutan de forma automatizada **63 pruebas unitarias e integradas**, alcanzando una tasa de éxito del **100%**.
- Se registra una cobertura del **86,3% en el código general** y un **85,8% enfocado estrictamente en el código nuevo**, superando el umbral estándar industrial recomendado de calidad (80%).
- La lógica condicional alcanza una cobertura de ramas del **76,5% en el código general** y **73,3% en el código nuevo**, asegurando la protección automatizada de los flujos alternativos y flujos de error en el ciclo de vida.

4. Instrucciones de Instalación (Desde Release de GitHub)

4.1 Prerrequisitos.

- **Docker:** Requisito para el método de ejecución recomendado, encargado de levantar los contenedores aislados de la aplicación y el motor relacional sin necesidad de configuraciones locales.
- **Java Runtime Environment (JRE) 17:** Requisito necesario únicamente si se opta por el método de despliegue directo

4.2 Método 1: Ejecución mediante Docker

Este enfoque automatiza por completo el aprovisionamiento de la infraestructura. El pipeline de GitHub descarga e integra de forma nativa la imagen precompilada del sistema y la enlaza a un contenedor PostgreSQL de producción.

Pasos para la instalación:

1. Descargar el archivo de configuración multi-contenedor docker-compose.yml desde el repositorio de GitHub.
2. Situar la consola de comandos en el directorio donde se ubica el archivo descargado y ejecutar el comando de inicialización: **docker compose up -d**
3. El motor descargará la imagen mgcss-track, validará la correcta disponibilidad de la base de datos PostgreSQL en el puerto 5432 y, posteriormente, levantará el contenedor de la aplicación.
4. Una vez completado el flujo, acceder de manera directa al panel de gestión web ingresando en el navegador la dirección: <http://localhost:9090/index.html>.

4.3 Método 2: Ejecución Local mediante .jar

Este enfoque permite ejecutar el servidor de la aplicación de forma nativa e independiente en el sistema operativo.

Pasos para la instalación:

1. Acceder al apartado de **Releases** dentro del repositorio del proyecto en GitHub y descargar el ejecutable con extensión .jar empaquetado automáticamente por el workflow de producción.
2. Validar que no existan restricciones de red o conflictos de puertos en el puerto local predeterminado de la aplicación (8080).
3. Lanzar la ejecución del servidor desde la terminal del sistema operativo invocando el comando: **java -jar app.jar** (Sustituir app.jar por el nombre exacto del archivo descargado de la release).
4. Al arrancar de forma aislada fuera del ecosistema Docker, el sistema se activará bajo el perfil de desarrollo y conectará de forma transparente con la base de datos **H2 en memoria**. Una vez inicializado, acceder a la web a través de la URL: <http://localhost:8080/index.html>.

5. Demo Funcional

5.1 Descripción de la interfaz web

La interfaz de usuario de MGCSS está diseñada como una aplicación de página única ligera y totalmente desacoplada, construida utilizando tecnologías estándares de la web: **HTML, CSS y JavaScript**.

Sus características principales son:

- **Simulador de Sesión por Roles:** En la cabecera se incluye un selector desplegable que permite alternar en tiempo real entre el rol de **Administrador** y los diferentes **Técnicos** registrados en el sistema.
- **Renderizado Condicional:** El script manipula de forma dinámica al cambiar de rol, muestra u oculta los botones de gestión y recalcula qué acciones tiene permitidas cada técnico en la tabla de solicitudes.
- **Comunicación Asíncrona (REST):** La interfaz no recarga la página, todas las operaciones se realizan de forma asíncrona mediante la API interactuando directamente con los endpoints expuestos por los controladores de Spring Boot.

5.2 Flujo Principal de uso.

A. Creación de Cliente y Técnico

1. **Registro de Cliente:** Actuando como **Administrador**, el usuario pulsa "+ Registrar Nuevo Cliente". La aplicación solicita el nombre mediante una ventana emergente.
2. **Registro de Técnico:** El Administrador pulsa "+ Registrar Nuevo Técnico", lo que dispara una llamada. Al completarse, el nuevo operador se añade automáticamente a la lista de técnicos disponibles en el simulador de sesión.

B. Registro y Ciclo de Vida de una Solicitud

1. **Apertura (estado ABIERTA):** El Administrador hace clic en "+ Nueva Solicitud", lo que despliega un formulario modal. Introduce la descripción de la incidencia (ej. "Fallo en servidor de correo") y selecciona al cliente solicitante desde un menú desplegable. Al presionar "Guardar", La tarea se registra e inicializa en el backend con la fecha actual y el estado ABIERTA.
2. **Asignación de Trabajo:** En la tabla principal, el Administrador visualiza la nueva solicitud en la columna de técnico como Sin asignar. Utilizando el selector de la columna "Acciones", elige uno de los técnicos activos del sistema. La capa de dominio valida que el técnico esté disponible y asocia la tarea.
3. **Procesamiento (estado EN_PROCESO):** Si el usuario cambia el rol del simulador al del técnico asignado, la interfaz se refresca condicionalmente. En lugar del desplegable de asignación, el técnico ve un botón verde etiquetado como "Iniciar". La solicitud cambia su estado a EN_PROCESO y añade la entrada correspondiente a su historial cronológico de auditoría.
4. **Finalización (estado CERRADA):** Con la tarea en marcha, el botón del técnico cambia a un botón azul llamado "Cerrar Tarea". Al hacer clic, el sistema solicita confirmación y envía una petición. El dominio evalúa la transición y, al cumplir la regla de negocio (provenir del estado de proceso), la convierte en CERRADA, inhabilitando más acciones sobre ella para el operario.
5. **Reapertura Opcional:** Si un **Administrador** vuelve a entrar al panel, verá que la solicitud finalizada muestra un botón de "Reabrir". Al pulsarlo, reinicia el ciclo regresando la tarea de forma segura al estado EN_PROCESO.